

The Fractal Design System

A recursive, self-improving level designer — and why it works. Status: **design** (the foundation is shipped; the loop is not yet built). Companion to [docs/MECHANICS-PIPELINE.md](#).

The idea in one breath

Game content **nects**: a *move* composes into a *room*, rooms into a *level*, levels into a *campaign*. If we make the two core design acts — **evaluate** (how good is this thing?) and **improve** (make it better) — *the same shape at every scale*, then one small engine drives the whole ladder, and it can **recurse**: improve a campaign by improving its levels by improving their rooms by improving their moves. The thing that makes any of this possible — and that almost every procedural system lacks — is a **deterministic, mechanics-aware fitness oracle**: a function that, given a piece of a level, tells you objectively whether it works and how hard it is. We already built that (the verb-aware solver). This document explains the concept, layers a **BSP/contract-tree** representation on top of it, and — most importantly — explains *why each piece works*.

Part 1 — The fractal: one loop, every scale

The scale ladder

```
campaign — ordered levels ————— good arc? variety across biomes?
  level   — ordered rooms ————— rising difficulty curve? teaches then tests?
    room  — a graph of moves ————— solvable? does it REQUIRE the dash? how tight?
      move — run/jump/dash/walljump ——— lands where intended? how much margin?
```

Each rung is a **container of the rung below**. The fractal claim is simply: the operations *evaluate* and *improve* have the **same signature** at every rung. Evaluation flows **up** (a level's score is computed from its rooms' scores). Improvement flows **down** (to fix a level, fix or re-order its rooms).

The universal loop

At every scale, designing is the same three-step loop, iterated:

generate → **evaluate** → **select**

- **generate** a candidate (mutate something that exists, or synthesize from the spec),
- **evaluate** it into an objective *fitness vector*,
- **select** the better one (hill-climb / evolutionary search), subject to hard *gates*.

We write that loop **once** — `evolve(seed, { mutate, fitness, target, gate, budget })` — and instantiate its `mutate` and `fitness` per scale. Room `fitness` = the solver. Level `fitness` = the difficulty curve over its rooms. Same engine, three rungs.

Why the fractal works

Three reasons, and they're the whole argument:

1. **Write-once reuse.** A search loop is the same algorithm regardless of what it's searching over. Once you've paid for it, every scale is free — you only supply a small `mutate` and a `fitness` per rung. 2. **Cheap recursion.** A higher rung's fitness is computed *from the lower rung's already-objective numbers* — a level's quality is a function of its rooms' (already measured) difficulties. So evaluating the top of the ladder doesn't re-derive the bottom; it aggregates it. Recursion that composes is cheap. 3. **It mirrors how humans actually design.** Outline → sections → scenes; or "first the cave, *then* the cliff," then fill each in. Coarse-to-fine with local refinement is the natural shape of design, and a fractal loop is exactly that shape made mechanical.

Part 2 — The keystone: the fitness oracle

Everything stands on one function being trustworthy:

```
solveRoom(room, spec) → { solvable, requires:[verbs], usedVerbs, tightness, difficulty, path }
```

It forward-simulates the hero through a single-screen room (real physics, `platforming.mjs`) and reports: is it solvable with the declared verbs; which verbs are **load-bearing** (remove the dash → is it still solvable? if not, the room *requires* the dash); how close to the spikes the only line runs (tightness); and a difficulty score. It even emits the autopilot path the game replays.

Why the oracle works (and why it's the rare part)

A search loop is only as good as its fitness function. Most procedural generators die here: they can *make* a thousand levels but can't *tell which one is good*, so they fall back to hand-authored rules or a human in the loop. Ours is different because the oracle is:

- **deterministic** — pure simulation, no randomness, no rendering → the **same room always scores the*

*same**, so search is stable and a good result is reproducible forever;

- **objective** — it answers yes/no and gives numbers, not vibes;
- **cheap** — a single screen solves in about a second, so a search can try thousands of candidates;
- **mechanics-aware** — it reasons about the *actual* run/jump/dash/wall-jump/spring chains, so "this room

requires the dash" is a fact about the game, not a proxy.

That last property is the moat. It's the difference between a critic that says "this feels hard" and one that *proves* "a player without the dash literally cannot finish this room." You can optimize honestly against the second; you can't against the first.

Fitness is a vector, gated, never traded

A single "fun number" is too thin to optimize. Fitness is a **vector** — solvability + required-verbs + tightness + difficulty (solver), felt-fun (`feelmodel`), reward reachability (`reach`), curve-fit (`difficulty`). It collapses to a scalar only at the moment of selection, as *distance to a target band* ("requires dash; difficulty 60–80; tightness 40–60"). And some constraints are **hard gates** — never accept an unsolvable room, a room below the fun floor, or a stranded reward, no matter how good the rest of the score is. The rule is **AND, not trade**: a room must be solvable *and* demand its verb *and* be fun. The optimizer only moves *within* that feasible set, toward the target. (This discipline already exists in `funmax.mjs` ; the fractal version just generalizes it.)

Part 3 — BSP and contracts: the representation that lets an LLM evolve the design

The fractal loop says *how to search*. BSP says *what to search over* — and it's the piece that turns "an LLM flails at a whole level" into "an LLM reliably improves the design."

A level is a binary tree of regions

Borrowing from how roguelikes have generated dungeons for decades: instead of a flat list of rooms, a level is a **BSP tree** — recursively **cut the whole journey in two** until each leaf is one screen. In a vertical climber the cut is a horizontal line at some height; in a horizontal game it's a vertical line. The **in-order traversal of the tree is the play sequence**.

The cut is "a wall with one door," and the door is a contract

This is the load-bearing idea. When you cut a region in two, you also place **one gateway** between the halves. That gateway isn't just geometry — it's a **contract**:

the player crosses here, at this position, carrying this state (dash available? at this height? this much momentum?), and this cut divides the difficulty budget into a lower share and an upper share.

That's a **precondition / postcondition for the node** — Hoare logic for level design. Each subtree's job becomes small and provable: *given the entry state at my bottom door, deliver the player to my top door in the exit state, spending my budget*.

```

Node = {
  region:    { x0, y0, x1, y1 },                // the slab this node owns
  contract: { enterAt, enterState, exitAt, exitState, budget, theme },
  cut:      { axis, at, door } | null,          // internal: how it split + the gateway
  children: [Node, Node] | null,                // internal
  room:     Room | null,                       // leaf: solveRoom-verifiable against
  seed
}

```

Why the contract-tree works

1. Compositional verification — you never solve the whole level. The solver can certify a single screen, but it chokes on a long monolith (this is exactly why Vesper Peak's 10-move face had to be hand-split into links). The contract-tree makes that split the architecture: verify each **leaf** against its contract; the **doors compose** (a child's postcondition is its parent's internal boundary is the sibling's precondition); therefore *a tree of verified leaves with matching contracts is a verified level*. Whole-level correctness with only ever screen-sized proofs. The level-scale verifier I was missing isn't a bigger solver — it's *contract-matching over the tree*.

2. Locality — edits stay local, so evolution is tractable. In a flat room list, editing room 3 ripples through the whole difficulty curve. In the tree, each subtree owns a bounded region *and* a bounded slice of the budget, and the door is its *only* interface. So you can re-roll one subtree and the rest is *provably untouched*. "Make the top third harder, still requiring wall-jumps" = regenerate one subtree against a tighter budget; contracts preserved; everything else still verified. Bounded blast radius is what makes iterative improvement safe.

3. One tree, three meanings — the difficulty curve becomes free. The same binary tree simultaneously encodes **space** (the partition), **play order** (in-order traversal), and **difficulty budget** (each cut allocates the budget between its children). So pacing stops being a thing you separately score and becomes a thing that *emerges* from how budget flows down the cuts — "late climax at ~84%" is just "weight budget toward the late subtrees." Lumpy pacing is fixed by a **tree rotation**, like rebalancing a search tree, with the curve-fit fitness telling you which rotation helps.

4. The tree is an abstraction ladder — reason at any altitude. You can talk about "the ice section" as one node without expanding it, or zoom into a single room. The LLM (and the human) can plan top-down (allocate the act structure) then fill bottom-up — exactly how good designers and writers work.

Why this specifically lets an LLM evolve the design

Here's the crux. Asking an LLM to "design a good five-screen level" fails: it can't hold the whole thing, can't verify it, and gets no ground truth. The tree + contracts change the question it's asked:

"This region is the ice section. The player enters bottom-left with the dash, must exit top-right launched off a spring, budget 45–55, theme ice. Either **cut it** — propose the dividing line, how to split the budget, and where the door goes — **or fill it as a leaf** — propose the room geometry and the idea verb."

That is a **small, well-typed, locally-verifiable decision** — the kind LLMs are *good* at — and `solveRoom` hands back **precise, local feedback**: "solvable, but the gap is 130px so it doesn't *require* the dash; the contract demanded it; widen to ≥ 150 px." Tight loop, ground truth, bounded scope. So the architecture is **recursive-descent generation with the LLM as the per-node oracle and the solver as the verifier** — the LLM is the *intelligent mutation operator*, replacing blind random nudges, and the solver is the selection pressure.

And because the **tree is the genome**, evolution gets real operators: *re-cut* a node, *regenerate* a leaf (new room, same contract), *swap or rotate* subtrees, **graft** a great room from a library, and **crossover** two levels by swapping subtrees *wherever their contracts match*. That's genetic programming on level-trees — a far richer search than mutating a flat list. Binary splits keep each decision minimal (one yes/no, one door), so a whole level is $O(\text{rooms})$ trivial decisions instead of one impossible one.

Part 4 — Honesty: the seam that must close first

The solver simulates `platforming.mjs`; the live game plays its own `game.js`. They're *validated to match*, not *unified* — and an optimizer that hill-climbs hard against the solver could converge on something the player doesn't quite feel. **Fix before trusting deep recursion**: lift the movement into one shared `Studio.Platforming` that the game drives *and* the solver simulates, so "what the solver proves" *is* "what the player feels" by construction. Until then, every generated level still has to pass the real `eval.mjs` gate (which runs the solver's path in the actual game) — **the loop proposes; the real gate disposes**.

Two more honest limits: *difficulty is not fun* (mitigated by AND-ing the felt-fun signal into the fitness), and *the fractal shape is genre-agnostic but the oracle is not* — each new genre needs its own solver (its own physics simulator) plugged into the same loop. That's the price of generalizing past platformers.

Part 5 — What exists, and the smallest next step

piece	status
the spec (<code>mechspec.mjs</code>), the movement sim (<code>platforming.mjs</code>)	✓
the fitness oracle (<code>solver.mjs</code> → <code>solveRoom</code>)	✓
the composer (<code>rooms.mjs</code> → <code>chainRooms</code>), the retrofit auditor (<code>upgrade-mechanics.mjs</code>)	✓
a <i>precedent</i> loop, gated (<code>funmax.mjs</code> / <code>evolve.mjs</code> , old format)	✓
the room generator (<code>mutateRoom</code>)	✗
the BSP/contract tree + compositional verifier	✗
the generic <code>evolve</code> loop + LLM node-oracle	✗
movement unification (<code>Studio.Platforming</code>)	✗

The one move that flips "we can measure levels" into "we can recurse and improve them" is small: a `mutateRoom` operator + an `evolveRoom(seed, spec, target)` hill-climber that uses `solveRoom` as fitness. Concretely:

*Ask: generate a room that **requires the dash**, difficulty 60–80, tightness 40–60, reproducible from seed 42. Loop: start from a seed room → `mutateRoom` (nudge a ledge, add a spike, shift the take-off) → `solveRoom` scores it → keep iff closer to target and the gates hold → repeat. Out: a verified, reproducible room with its autopilot path, ready to drop into `chainRooms` and the real gate.*

From there: wrap it as the generic `evolve` , add the BSP tree so the LLM fills contracted nodes, score the level arc as budget-down-the-cuts, and unify the movement. At that point the engine isn't a pipeline that *builds* levels — it's a fractal that can **recurse and improve** them, with an LLM as the designer at every node and the solver as the judge.

Why it works, in one line

A **deterministic, mechanics-aware oracle** gives you honest feedback; a **fractal loop** reuses one search at every scale; and a **BSP/contract tree** cuts the level into small, verifiable, locally-editable nodes — so an LLM can do what it's good at (design one contracted piece) while the solver does what it's good at (prove the whole thing holds together).